SAND LOGIX

ShamCash

E-Payment
Document

Preparation Date

01/06/2026

Version

1.0.0

Change Log

Version	Date	Description of Changes
1.0.0	2026-06-01	Initial Release

1. Introduction **3**
2. Service Creation **3**
3. Full Payment Flow Overview **4**
3.1. Steps: 4
3.2. Sequence Diagram 6
4. General Request & Response Format **7**
5. Security & Encryption (Direct JWE) **8**
6. Endpoints **10**
6.1. Create Bill 10
6.2. Get Bill Info 12
6.3. Refund Bill 14
6.4. Get Transactions 16
7. Response Codes **19**
7.1. Application Codes 19
7.2. HTTP Status Codes 20
8. Redirect & Checkout Specification **21**
9. Webhook Specification **22**
10. Recommendations **23**
11. Reconciliation Approach **25**

1. Introduction

Electronic Payment enables third-party applications to initiate a seamless payment process directly within ShamCash. Using a deeplink URL, Agent systems can redirect users to ShamCash's payment interface to settle invoices quickly and securely.

Key Capabilities:

- Create a bill via API.
- Check the current bill status using its `billNo`.
- Redirect users to ShamCash for payment via a deeplink.

This feature ensures a smooth and consistent user experience by automating bill creation, presentation, and tracking.

2. Service Creation

Before integrating with ShamCash, each third-party application must complete a **Service Creation** step.

Information Required From the Third Party

- **ShamCash Verified Commercial Account** – A valid, active, and fully verified business account within the ShamCash
  You can create your commercial account online here
- **Logo URL** – An accessible HTTP URL for loading and displaying the partner's logo inside ShamCash.
- **Application Name** – The name that will appear to users when paying through ShamCash.

Information Provided to the Third Party

Once the service is created, ShamCash issues:

- **agentKey** – Identifies the partner application.
- **secretKey (AES Key)** – A 32-byte AES key.
- **baseUrl** – The root network address used to access the ShamCash API endpoints.

ShamCash Assets

To properly display ShamCash as a payment method on your website checkout page or within your native application layout, please use our official branding assets:

- **ShamCash Logo for Dark Backgrounds:** Official Dark SVG Asset
- **ShamCash Logo for Light Backgrounds:** Official Light SVG Asset

3. Full Payment Flow Overview

3.1. Steps:

1. Bill Creation

The Agent System initiates the process by sending a `POST /createBill` request to ShamCash containing the bill details, along with a `callbackUrl` (server-to-server webhook) and a `redirectUrl` (client-side browser redirection). Upon successful validation, ShamCash creates the bill record and returns a **`paymentUrl`** to the Agent.

2. User Redirection

The Agent System redirects the user's browser or application to the provided **`paymentUrl`**. This `URL` routes the user securely to the ShamCash hosted payment page or opens the native ShamCash App if installed on the device.

3. Bill Presentation

On the checkout interface, the user is presented with the clear, unalterable details of the bill (Biller Name, Logo, Exact Amount, and Description Notes).

4. Bill Payment Lifecycle & Exceptions

The user reviews the payment details and authorizes the transaction using their Transfer Confirmation Code. The flow handles three possible paths:

- **Success:** The transaction processes successfully, and the bill status transitions to `Paid`.
- **Failure:** The transaction is blocked due to business rules, and an explicit error code is displayed. Common exceptions include:
  - *Insufficient Balance*
  - *Account Blocked*
  - *Invalid Confirmation Code*
  - *Confirmation Code Not Initialized* (Setup required before payment authorization).
- **Ignore:** The user abandons the checkout session or closes the interface without executing the payment, and the bill will expire.

5. Client-Side Redirection (`redirectUrl`)

Immediately after the payment interaction ends, ShamCash redirects the user's browser tab or application context back to the Agent's specified **`redirectUrl`**.

6. Asynchronous Backend Notification (`callbackUrl`)

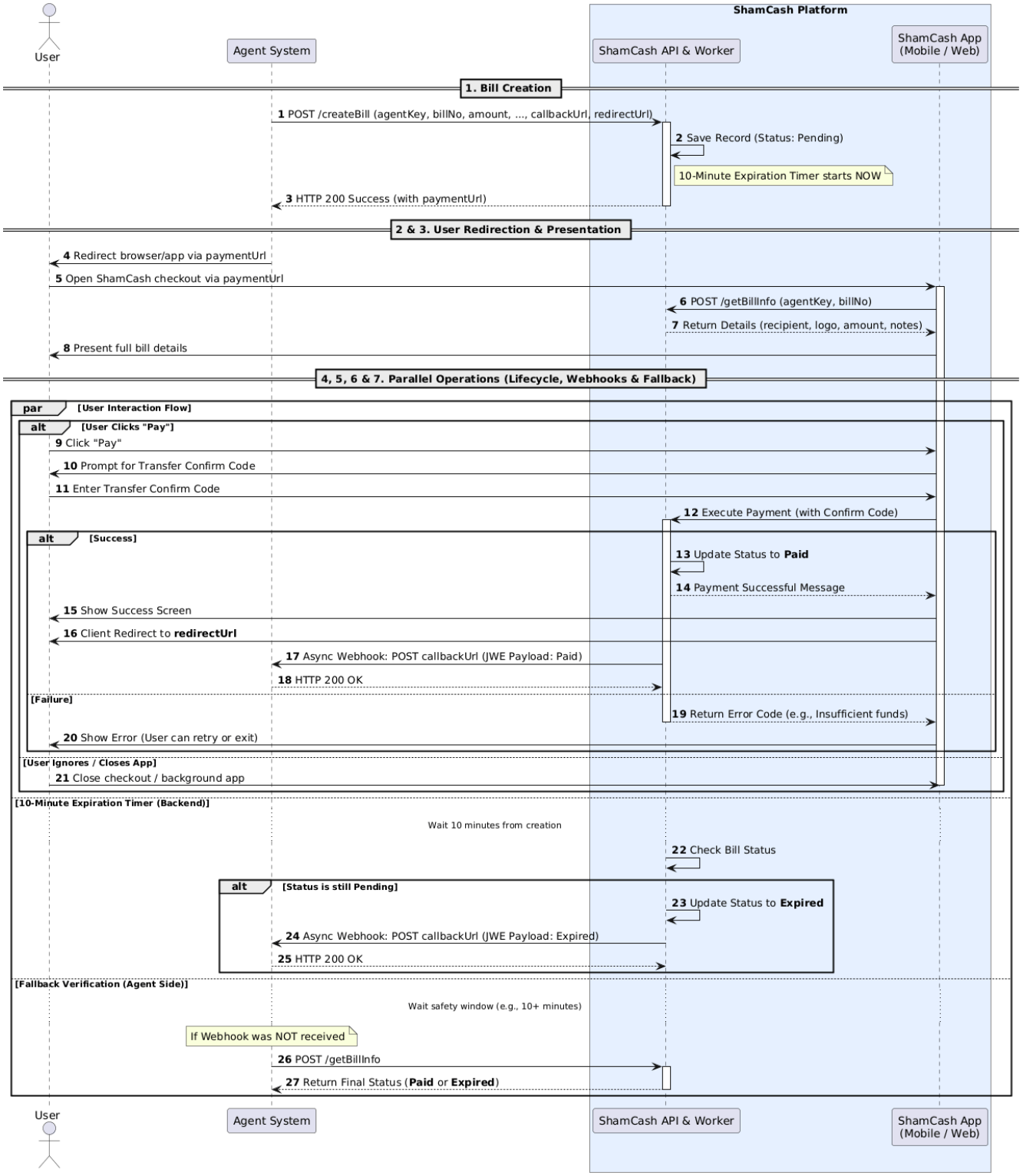
The moment a bill reaches a final terminal state—whether the operation succeeded or failed (such as `Paid` or `Expired`)—ShamCash automatically dispatches an asynchronous notification directly to the Agent's registered `callbackUrl`.

Each webhook request is atomic, sending exactly one distinct transaction event at a time. The Agent's server must process this notification independently and promptly return an HTTP `200 OK` status to confirm successful receipt.

7. Expiration & Fallback Reconciliation (`getBillInfo`)

- **Expiration Timer:** Operating in parallel from the moment the bill is created, a background timer runs on the ShamCash server. If the payment is not completed within 10 minutes, the bill automatically switches to an `Expired` state and is no longer valid.
- **Fallback Verification:** The Agent System uses the `POST /getBillInfo` endpoint solely as a fallback reconciliation mechanism. If the Agent does not receive the Webhook callback within a reasonable safety window (e.g., 10 minutes), they query this endpoint to verify the final status of the bill.

3.2. Sequence Diagram



4. General Request & Response Format

Request Body Structure

All payment APIs use the following base request structure:

```
{
    "encData": "STRING",
    "agentKey": "STRING"
}
```

Field Descriptions

- **agentKey**: A unique key provided to the third-party during service creation.
- **encData**: The secured payload, formatted as a standard Direct JWE token (AES-GCM 256-bit). It contains the encrypted request parameters and security claims (like expiration times).

Response Structure

All responses follow a unified format:

```
{
    "result": INT,
    "succeeded": BOOL,
    "data": null or OBJECT,
    "message": "STRING"
}
```

Field Descriptions

- **result**: An integer code representing the outcome of the operation. Each success or failure scenario has a unique result code.
- **succeeded**: A boolean value (`true` / `false`) indicating whether the request was successfully processed.
- **data**: A response object containing API-specific returned data. It may be null when no data is returned or when the operation fails.
- **message**: A descriptive text message, typically used to explain errors or provide additional context.

Note: Standard API responses are returned as cleartext JSON for immediate parsing, whereas incoming API requests and outbound webhook payloads strictly require JWE encryption.

5. Security & Encryption (Direct JWE)

ShamCash utilizes **Direct JSON Web Encryption (JWE)** using the `A256GCM` encryption algorithm. Every request payload must be encrypted, and every webhook payload you receive will be encrypted using this exact standard.

A standard JWE token consists of five separate parts, concatenated by a period (`.`):
`BASE64URL(Header) . BASE64URL(EncryptedKey) . BASE64URL(IV) . BASE64URL(Ciphertext) . BASE64URL(AuthenticationTag)`

Because ShamCash uses "Direct" encryption (a pre-shared symmetric key), the **EncryptedKey** segment is always left empty. The structure will look like this:
`Part1..Part3.Part4.Part5`

Mandatory Security Claims (Replay Protection)

To protect the integration from network manipulation and Replay Attacks, every JSON payload encrypted within the JWE **must** include the following standard time-based claims at the root level of the object:

- `iat` **(Issued At):** A standard Unix timestamp (in seconds) representing the exact moment the payload was generated by your system.
- `exp` **(Expiration Time):** A standard Unix timestamp (in seconds) representing the moment the payload becomes invalid.
    - **Best Practice:** We strictly recommend setting the `exp` to **5 minutes** after the `iat` time.
    - **Enforcement:** Any API request received where the current ShamCash server time is strictly greater than the `exp` timestamp will be immediately rejected.

Example

```
{
    "billNo": "BN-123",
    "amount": 10.50,
    "idempotencyKey": "9a3f8b7c-2e1d-4c5b-a6f7-e8d9c0b1a2f3",
    "note": "Customer requested a return for item A.",
    "iat": 1777643724,
    "exp": 1777644024
}
```

Encryption Steps (How to build `encData`)

1. Prepare the Secret Key

- The `secretKey` provided to you by ShamCash is a Base64-encoded string representing exactly 32 bytes of random data.
- Decode this Base64 string into its raw binary format to obtain the 32-byte (256-bit) encryption key. *(Do not use UTF-8 string encoding on the Base64 string itself).*

2. Prepare the JWE Header

- Create the exact JSON header required for Direct AES-GCM: `{"alg":"dir","enc":"A256GCM","cty":"json"}`
- Encode this JSON string using **Base64Url** (URL-safe Base64 with no `=` padding).
- Convert this Base64Url-encoded string into ASCII bytes. This will be used as the **Additional Authenticated Data (AAD)** during encryption.

3. Prepare the Payload & Security Claims

- Start with the JSON object containing your request parameters (e.g., `billNo`, `amount`).
- **Mandatory Claims:** You must inject `iat` (Issued At) and `exp` (Expiration Time) as Unix timestamps into the JSON. Tokens typically expire 5 minutes after creation to prevent replay attacks.
- Convert the final JSON string to a UTF-8 byte array (`plaintext`).

4. Generate the IV (Nonce)

- Generate a cryptographically secure, random **12-byte** nonce.
- Encode the nonce using Base64Url.

5. Execute AES-GCM Encryption

- Initialize the AES-GCM cipher using the raw 32-byte secret key and a 16-byte authentication tag size.
- Encrypt the `plaintext` using the 12-byte nonce and the Header's ASCII bytes as the `AAD`.
- This process yields two byte arrays: the `ciphertext` and a 16-byte `tag`.
- Encode both the `ciphertext` and the `tag` using Base64Url.

6. Assemble the Final Token

Concatenate the parts using a period (`.`) delimiter. Remember to leave the second part empty: `{EncodedHeader}..{EncodedIV}.{EncodedCiphertext}.{EncodedTag}`

6. Endpoints

6.1. Create Bill

POST: `{baseUrl}`/api/ElectronicPayment/createBill

Request:

`encData`: AES-GCM encrypted serialized object containing the following fields:

Key	Type	Description
billNo	STRING	Unique bill number generated by the agent system.
amount	DECIMAL(18, 2)	Amount of the bill to be created.
currencyId	INTEGER	Currency identifier 1 for USD, 2 for SYP
callbackUrl	STRING	A valid absolute HTTP/HTTPS URL. The webhook endpoint where ShamCash will send asynchronous status updates (e.g., Paid, Expired). Max length: 500 characters.
redirectUrl	STRING	A valid absolute HTTP/HTTPS URL. The endpoint where the user's browser will be redirected after finishing the payment session. Max length: 500 characters.
note	STRING?	Optional note or description related to the bill.

Example:

```
{
    "billNo": "BN-123",
    "amount": 10.50,
    "currencyId": 1,
    "callbackUrl": "https://agent-domain.com/api/webhook/shamcash",
    "redirectUrl": "https://agent-domain.com/checkout",
    "note": "Example...",
    "iat": 1777643724,
    "exp": 1777644024
}
```

Response (in success case):

`data`: object containing the following fields:

Key	Type	Description
statusId	INTEGER	The numerical identifier for the bill's status
statusName	STRING	The human-readable name of the transaction's status
billNo	STRING	Unique bill number generated by the agent

		system.
amount	DECIMAL(18, 2)	Amount of the bill to be created (Must be a positive value greater than or equal to `0.01`) .
currencyId	INTEGER	Currency identifier 1 for `USD`, 2 for `SYP`
createdDate	DATE	The bill create date
createdTime	TIME	The bill create time
tranId	LONG?	The sequential ShamCash transaction ID issued exclusively upon successful bill payment.
paymentUrl	STRING	The direct link generated by ShamCash to redirect the user to complete the bill payment.
rem	STRING	Remarks or notes related to the bill generated by the agent.
refunds	ARRAY	A collection of refund objects associated with this bill. Returns an empty array `[]` if no refunds have been processed.

Example:

```
{
    "result": 2500,
    "succeeded": true,
    "data": {
        "statusId": 1,
        "statusName": "pending",
        "billNo": "BN-123",
        "amount": 10.50,
        "currencyId": 1,
        "createdDate": "2026-05-19",
        "createdTime": "14:37",
        "paymentUrl": "https://.....",
        "rem": "Example...",
        "tranId": null,
        "refunds": []
    },
    "message": "Success"
}
```

Note

- Reusable Structure: The `data` object perfectly matches the GetBillInfo response structure to simplify your model mapping.
- Fixed Creation States: Since the bill is newly created, it always initializes with:
  - `"statusId": 1` / `"statusName": "pending"`
  - `"tranId": null`
  - `"refunds": []`
- Required Action: Your system must extract the `paymentUrl` and redirect the user to it to complete the payment within ShamCash.

6.2. Get Bill Info

POST: `{baseUrl}`/api/ElectronicPayment/getBillInfo

Request:

`encData`: AES-GCM encrypted serialized object containing the following fields:

Key	Type	Description
billNo	STRING	Unique bill number generated by the agent system.

Example:

```
{
    "billNo": "BN-123",
    "iat": 1777643724,
    "exp": 1777644024
}
```

Response (in success case):

`data:` object containing the following fields:

Key	Type	Description
statusId	INTEGER	The numerical identifier for the bill's status
statusName	STRING	The human-readable name of the transaction's status
billNo	STRING	Unique bill number generated by the agent system.
amount	DECIMAL(18, 2)	Amount of the bill to be created.
currencyId	INTEGER	Currency identifier 1 for USD, 2 for SYP
createdDate	DATE	The bill create date
createdTime	TIME	The bill create time
tranId	LONG?	The sequential ShamCash transaction ID issued exclusively upon successful bill payment.
paymentUrl	STRING	The direct link generated by ShamCash to redirect the user to complete the bill payment.
rem	STRING	Remarks or notes related to the bill generated by the agent.
refunds	ARRAY	A collection of refund objects associated with this bill. Returns an empty array [] if no refunds have been processed.

Refund Object Parameters (Inside data.refunds)

Key	Type	Description
tranId	LONG	The unique, sequential ShamCash transaction identifier generated specifically for this refund movement.
amount	DECIMAL	The specific financial amount returned to the user in this refund instance.
createdDate	DATE	The date when this specific refund was executed (yyyy-MM-dd).
createdTime	TIME	The time when this specific refund was executed (HH:mm).
rem	STRING	The reason or note provided for this specific refund transaction.

Example:


```
{
    "result": 2500,
    "succeeded": true,
    "data": {
        "statusId": 5,
        "statusName": "partly refunded",
        "billNo": "BN-123",
        "amount": 5000.00,
        "currencyId": 2,
        "createdDate": "2026-05-19",
        "createdTime": "15:34",
        "paymentUrl": "https://.....",
        "rem": "Test...",
        "tranId": 3827551,
        "refunds": [
            {
                "tranId": 3919459,
                "amount": 50.11,
                "createdDate": "2026-05-25",
                "createdTime": "12:43",
                "rem": "test refund 1"
            },
            {
                "tranId": 3919460,
                "amount": 1000.00,
                "createdDate": "2026-05-25",
                "createdTime": "12:46",
                "rem": "test refund 2"
            }
        ]
    },
    "message": "Success"
}
```

6.3. Refund Bill

POST: {baseUrl}/api/ElectronicPayment/refundBill

Purpose: This API allows the agent system to initiate a full or partial refund for a previously paid bill. The refunded amount is processed as a new financial transaction and credited back directly to the user who paid the bill.

Request:

encData: AES-GCM encrypted serialized object containing the following fields:

Key	Type	Description
billNo	STRING	The unique bill number of the original paid transaction to be refunded.
amount	DECIMAL(18, 2)	The specific amount to be refunded in this request. Must strictly enforce a maximum of 2 decimal digits.
idempotencyKey	STRING	A unique, cryptographically secure identifier generated by the agent to prevent duplicate operations. **Length:** 10 to 100 characters.
note	STRING?	Optional reason or remarks explaining why this refund is being executed.

Example:

```
{
    "billNo": "BN-123",
    "amount": 10.50,
    "idempotencyKey": "9a3f8b7c-2e1d-4c5b-a6f7-e8d9c0b1a2f3",
    "note": "Customer requested a return for item A.",
    "iat": 1777643724,
    "exp": 1777644024
}
```

Response (in success case):

data: object containing the following fields:

Key	Type	Description
tranId	LONG?	The unique ShamCash transaction ID issued exclusively for this specific refund movement.

Example:

```
{
   "result": 2500,
   "succeeded": true,
   "data": {
       "tranId": 3919468
   },
   "message": "Success"
}
```

Notes

- **Success-Only Idempotency Logic:** To protect against network instability and accidental double-refunds, the ShamCash gateway implements a strict Success-Only Idempotency strategy:
    - **The Happy Path:** If a refund request succeeds, the `idempotencyKey` is permanently locked within ShamCash for that specific Biller. If the Agent sends a subsequent request with the exact same key, ShamCash will not execute a new transfer. Instead, it bypasses the core financial ledger and safely echoes back the cached original success response containing the existing `tranId`.
    - **The Failure Path:** If a request fails due to business rules (e.g., *Insufficient Agent Balance*, *Invalid BillNo*) or hits a network timeout before reaching ShamCash's internal transaction ledger, the key is not locked. The Agent can safely resend the exact same request using the same `idempotencyKey` once the underlying issue is resolved.
- **Flexible Multi-Refunds:** The agent can execute a **Full Refund** (one-time return of the total bill value) or multiple **Partial Refunds** (splitting the amount across more than one request over time).
- **Strict Upper Bound Ceiling:** The aggregate sum of all processed partial refunds for any single `billNo` **must not exceed** the original total amount paid. Any request pushing the total refunded balance higher than the original bill value will be flatly rejected.
- **Implicit Currency Mapping:** The API automatically inherits the original bill's `currencyId` (e.g., USD or SYP). The agent cannot specify a different currency; refunds are always settled using the exact same asset type as the initial payment.
- **Status Updates:** Successfully triggering a refund will automatically alter the original bill's status when checked via `getBillInfo`—moving it to `partly refunded` (Status `5`) or `refund` (Status `2`) depending on the total balance returned.

Bill Status Codes

Status id	Status Name	Description
1	pending	Bill is awaiting user payment.
2	refund	The bill payment has been fully refunded back to the user.
3	expired	Bill expired (not paid within 10 minutes).
4	paid	Bill successfully paid.
5	partly refunded	A portion of the paid amount has been refunded, but **not the full value**.

6.4. Get Transactions

POST: `{baseUrl}`/api/ElectronicPayment/getTransactions

Purpose: This API returns a unified financial log of **all processed transactions** (both successful bill payments and executed refunds) associated with the agent. It utilizes **Cursor-based pagination** relying on the transaction ID (`afterTranId`) to efficiently fetch large sets of ledger data sequentially without performance degradation.

Request:

`encData`: AES-GCM encrypted serialized object containing the following fields:

Key	Type	Description
fromDate	DATE?	Optional starting date filter. Must strictly follow the **`yyyy-MM-dd`** format (e.g., `2026-05-23`). Defaults to `null`.
toDate	DATE?	Optional ending date filter. Must strictly follow the **`yyyy-MM-dd`** format (e.g., `2026-05-30`). Defaults to `null`.
afterTranId	LONG	cursor for pagination. Pass a transaction ID to fetch records created *after* this specific ID. Must contain **digits only**. Defaults to `0`.
limit	INT	The maximum number of records to return in a single response. Must contain **digits only** and be a value between `10` and `2500`. Defaults to `500`.

Example:

```
{
   "fromDate": "2026-05-01",
   "toDate": "2026-06-01",
   "afterTranId": 0,
   "limit": 10,
   "iat": 1777643724,
   "exp": 1777644024
}
```

Response (in success case):

data: object containing the following fields:

Key	Type	Description
hasMore	BOOLEAN	Indicates if there are additional paid records remaining to be fetched based on your query criteria.
lastReturnedTranId	LONG	The transaction ID (`tranId`) of the very last item in the current `list`. Use this value as the `afterTranId` in your next request to fetch the next batch of records.
list	ARRAY	A sequential collection of transaction objects containing specific details for each individual payment or refund action.

Transaction Object Parameters (Inside `data.list`)

Each item inside the list array represents a distinct financial movement and contains the following fields:

Key	Type	Description
billNo	STRING	Unique bill number generated by the agent system associated with this transaction.
amount	DECIMAL	The total financial value settled in this specific movement.
currencyId	INTEGER	Currency identifier: 1 for USD, 2 for SYP.
createdDate	DATE	The date when this specific transaction was executed (yyyy-MM-dd).
createdTime	TIME	The time when this specific transaction was executed (HH:mm).
tranId	LONG	The unique, sequential ShamCash system transaction identifier used for tracking and cursor-based pagination.
tranTypeId	INTEGER	The numerical identifier for the type of financial movement: 1 for Payment, 2 for Refund.
tranType	STRING	The human-readable name of the transaction type: "payment" or "refund".

Example:


```
{
    "result": 2500,
    "succeeded": true,
    "data": {
        "list": [
            ...,
            {
                "billNo": "Test-123",
                "amount": 5000.00,
                "currencyId": 2,
                "createdDate": "2026-05-19",
                "createdTime": "07:29",
                "tranId": 3827711,
                "tranTypeId": 1,
                "tranType": "payment"
            },
            {
                "billNo": "Test-156",
                "amount": 100.00,
                "currencyId": 1,
                "createdDate": "2026-05-21",
                "createdTime": "12:23",
                "tranId": 3919398,
                "tranTypeId": 1,
                "tranType": "payment"
            },
            {
                "billNo": "Test-156",
                "amount": 100.00,
                "currencyId": 1,
                "createdDate": "2026-05-24",
                "createdTime": "14:05",
                "tranId": 3919445,
                "tranTypeId": 2,
                "tranType": "refund"
            }
        ],
        "hasMore": true,
        "lastReturnedTranId": 3919445
    },
    "message": "Success"
}
```

7. Response Codes

7.1. Application Codes

Code	Message	Description
1000	General Error	Unexpected internal server error.
1304	Balance not enough	The agent's wallet balance is insufficient to cover the requested refund amount.
1322	The receiver is blocked from receiving	The user's account is restricted or blocked from receiving funds, preventing the refund from being credited.
1700	EPay Custom Error	Dynamic validation or payload error. The specific reason (e.g., missing data or missing keys)
1701	Agent Key Invalid	Authentication failed; the provided `agentKey` is incorrect.
1702	Agent Not Active	The agent has been deactivated (must contact ShamCash Support Team).
1703	Bill Not Found	The requested `billNo` does not exist for this `agentKey`.
1704	Bill No Already Exists	This billNo was already used.
1705	Encrypted Data is invalid	The request data (`encData`) is corrupt, tampered with, expired, or used an invalid key.
1706	Your Account is blocked	Your ShamCash account is blocked (must contact ShamCash Support Team)
1707	Origin Bill is not paid	The refund request was rejected because the original bill is not in a paid or refundable state.
1708	Refund amount exceeds the remaining bill balance	The requested refund amount is greater than the remaining non-refunded balance of the original bill.
2500	Success	

7.2. HTTP Status Codes

The ShamCash API distinguishes between **HTTP Transport Statuses** (network/routing) and **Application Response Codes** (business logic).

Please implement your integration handling based on the following behavior:

HTTP Status	Trigger Condition	Expected Payload
200 Ok	The request successfully reached the ShamCash server, the URL is correct, and a valid JSON body was sent.	**Always contains the JSON payload** (e.g., `result: 1700`, `result: 1704`, etc.). All business logic successes and failures are handled here.
400 Bad Request	The request payload contains malformed or structurally invalid JSON	Standard API gateway error response detailing the parsing failure.
404 Not Found	The API endpoint URL is incorrect or misspelled.	Standard gateway error.
415 Unsupported Media Type	The request body is empty, missing, or not sent as a valid `application/json`.	Standard gateway error.
500 / 502 Server Error	The ShamCash service is temporarily down or undergoing maintenance.	Gateway / Reverse Proxy error page.

Developer Note: Do not write code that assumes an HTTP `200 OK` means the transaction succeeded. An HTTP `200` simply means the server processed the request. You must parse the internal JSON body and inspect the `result` field (e.g., `1000`-`1705`) to determine the actual outcome of the operation.

8. Redirect & Checkout Specification

The integration system must simply redirect the user to the unique paymentUrl received in the `createBill` success response.

App vs. Web Redirection Behavior

- **App Redirection (Seamless Handshake):** If the ShamCash mobile application is installed on the user's device, clicking or redirecting to the `paymentUrl` will automatically trigger a **Deep-link**, opening the bill checkout screen directly inside the app.
- **Web Fallback (Secure Checkout):** If the ShamCash application is not installed, the `paymentUrl` will gracefully degrade and open the **ShamCash Web Portal** in the user's browser, allowing them to complete their payment securely via the web interface.

Note: The automatic mobile application deep-linking feature is strictly supported on **ShamCash App Version 2.2.5 or higher**. It is highly recommended to advise users to keep their mobile application updated to guarantee a seamless app-switch experience.

Checkout Flow & Actions

Once the user lands on the checkout interface (App or Web) via the `paymentUrl`, the ShamCash platform handles the following lifecycle:

1. **Verification:** The platform automatically decodes and validates the secure parameters embedded within the URL.
2. **Presentation:** It fetches the associated bill (ensuring its status is `pending`) and presents full details to the user, including the recipient's identity, business logo, amount, currency, and original notes.
3. **User Decisions:**
    - **Confirm & Pay:** The user authorizes the transfer using their Transfer Confirm Code then the system processes the funds immediately and shifts the status to `paid` (Status `4`).
    - **Ignore or Cancel:** The user dismisses the screen or closes the window then no funds are moved, and the bill remains pending until it automatically hits its 10-minute timeout and shifts to `expired` (Status `3`).

9. Webhook Specification

Unlike other endpoints where your system calls ShamCash, **this is an API endpoint that you must implement, host, and expose on your own server**. ShamCash will act as the client and actively invoke `callbackUrl` to push data to your system.

When a bill reaches a final terminal state—either **Paid** or **Expired**—the ShamCash backend dispatches an asynchronous HTTP `POST` request directly to the Agent's pre-registered `callbackUrl`.

Request Structure

The payload sent to your server is wrapped inside the standard unified structure. The actual business metrics are encrypted within a Direct JWE token to guarantee data integrity and security.

```
{
    "encData": "STRING"
}
```

Decrypted Webhook Payload Fields

Once your server successfully decrypts the `encData` token using your `SecretKey`, the resulting JSON object will contain the following structural properties:

Key	Type	Description
billNo	STRING	The unique bill number originally generated by your Agent System.
tranId	LONG?	The unique ShamCash financial transaction ID. This field is populated **only** when the payment succeeds; it returns `null` if the bill expires.
statusId	INTEGER	The numerical identifier representing the final state of the bill: • `4` = Paid (Success) • `3` = Expired
iat	LONG	Standard Unix timestamp (in seconds) indicating when this notification was generated.
exp	LONG	Standard Unix timestamp (in seconds) indicating when this token expires (set to 5 minutes after `iat`).

Example

Success Bill


```
{
    "billNo": "BN-123",
    "tranId": 3580921,
    "statusId": 4,
    "iat": 1777643724,
    "exp": 1777644024
}
```


Expired Bill


```
{
    "billNo": "BN-123",
    "tranId": null,
    "statusId": 3,
    "iat": 1777643724,
    "exp": 1777644024
}
```


Expected Agent Server Response

Your endpoint must validate the JWE token, check the exp timestamp to prevent replays, process your internal logic (such as updating order states or releasing locked inventory), and return an **HTTP 200 OK** status code within a **10-second safety window**.

Idempotency Notice

Due to potential network fluctuations, ShamCash might retry sending the same webhook event. Your system **must** be idempotent. Always inspect the billNo and ensure you do not process state mutations or credit logic more than once for the same transaction event.

10. Recommendations

Just-In-Time Bill Creation

- **Recommendation:** Do not create the bill in ShamCash when the order or cart is initially generated in your system. Instead, trigger the `POST /createBill` request only when the user **explicitly** clicks the final "Pay/Proceed to ShamCash" button.
- **Rationale:** This ensures that the 10-minute expiration timer on the ShamCash backend only starts when the user is actually ready to pay. Generating the bill too early means the countdown might finish and expire before the user completes the checkout process, forcing your system to unnecessarily generate a brand-new bill.

Decoupling Internal Order IDs from `billNo`

- **Recommendation:** Avoid mapping your internal primary Order ID exactly 1:1 to the ShamCash `billNo`. We recommend appending a retry suffix or using a sub-sequence format (e.g., `Order-123-1`).
- **Rationale:** If a payment attempt fails due to an error or timeout, you can easily generate a fresh `billNo` for the exact same order in your system without getting blocked by error code `1704 (Bill No Already Exists)`.

Network Interruption Handling

- **Recommendation:** If a network drop occurs during a `POST /createBill` request and the API subsequently returns error code **`1704 (Bill No Already Exists)`** upon retry, your system should intercept this gracefully.
- **Rationale:** Receiving a `1704` in this specific scenario confirms that your original request reached our servers and the bill was recorded successfully. Your system can safely bypass regeneration and transition straight into the **Polling** phase using that `billNo`.

Protect Your Secret Key (AES Key)

- **Recommendation:** Treat your `secretKey` (AES Key) with the highest level of confidentiality. Store it securely within server-side environment variables or a dedicated key vault system, and never expose it in client-side applications (like mobile apps or front-end JavaScript). If you suspect the key has been compromised, leaked, or misplaced, you must contact the ShamCash Support Team immediately to revoke and reissue it.
- **Rationale:** This key authorizes your entire system and encrypts payload data. A compromised key allows unauthorized actors to spoof transactions and access sensitive bill infrastructure under your agent profile.

Avoid Polling

Do not implement continuous polling loops using the `POST /getBillInfo` API. Regularly spamming this endpoint to check if a bill is paid causes unnecessary network overhead and degrades performance for both your system and ShamCash.

The Correct Flow:

1. Trust and rely entirely on the asynchronous **Webhook Callback** to receive real-time updates for `Paid` or `Expired` statuses.
2. Treat the `POST /getBillInfo` endpoint strictly as a **Fallback Reconciliation Mechanism**.
3. Only invoke this endpoint **once**, after a safety window of **10 minutes or more** from the bill's creation, and *only* if your server failed to receive the expected webhook notification

Redirect Handling - Use Default Browser, Not WebView

- **Recommendation**: When redirecting users to the ShamCash payment page, always open the URL in the device's default system browser (Chrome/Safari) rather than an embedded WebView.
- **Rationale**: WebViews don't share browser sessions (causing redundant logins), and break deep linking into the ShamCash mobile app—all of which increase payment abandonment and security risks.

User Education & Video Guides

- **Recommendation:** Add a short video tutorial or clip on your checkout page showing how to pay via ShamCash.
- **Rationale:** Shifting between apps can confuse users. A quick visual guide reduces confusion and increases successful payments.

11. Reconciliation Approach

To perform End-of-Day (EOD) or Monthly reconciliation, follow this pattern to ensure you retrieve all records:

1. **Initial Request:** Send the date range with `afterTranId: 0` and a reasonable `limit` (e.g., 1000).
2. **Check Response:** If the response contains `hasMore: true`, read the `tranId` of the **last record** in the list.
3. **Next Page:** Send the *same* request again, but update `afterTranId` to that last `tranId`.
4. **Repeat:** Continue this loop until `hasMore` is `false`.

Example 1: Initial Reconciliation Request

Goal: Get all paid bills for the first 20 days of January.

```
{
    "fromDate": "2026-01-01",
    "toDate": "2026-01-20",
    "afterTranId": 0,
    "limit": 1000
}
```

Example 2: Fetching the Next Page

Goal: Assuming the previous request returned 1000 records and the last transfer had `afterTranId: 8500`, the next request would be:

```
{
    "fromDate": "2026-01-01",
    "toDate": "2026-01-20",
    "afterTranId": 8500,
    "limit": 1000
}
```

12. Testing & Sandbox Environment

To ensure a seamless integration before moving to production, a dedicated testing environment is provided:

Credentials & Endpoints: The ShamCash Support Team will supply your technical team with the testing `Endpoints BaseUrl`, alongside a dedicated `AgentKey` and `SecretKey` (as detailed in the *Service Creation* section).

Mock Payment Flow: When utilizing the testing environment, the generated `paymentUrl` will redirect users to a **Sandbox Payment Interface**.

Test Confirmation Code: You can successfully authorize and simulate any test payment on this interface by entering the universal mock confirmation code: `000000`.